

Chat With Unknown Data Architecture & Feature Removal Guide

A code-based explanation of every application and pipeline layer, what each one does, how data moves between them, and what happens if optional features are disabled or removed.

Repository reviewed: **chat-with-unknown-data**

Prepared 19 August 2026

Scope: current source code in [api/](#), [pipeline/](#), [frontend/](#), configuration, caches, logging, and tests.

1. Executive summary

The system is a single-process, zero-shot Text-to-SQL application. A browser sends a natural-language question to FastAPI. The server discovers the active database structure, optionally enriches its meaning, retrieves a compact relevant schema, asks a Groq chat model to create T-SQL, executes and repairs the query, then asks the model to turn rows into a readable answer.

```
Browser UI →
FastAPI / intent routing / session state →
metadata shortcut? —yes→ direct structural answer ↳no→ [1 Introspection] →
[2 Enrichment] →
[3 RAG] →
[4 SQL generation] →
[5 Execute + self-correct] →
[6 Answer generation] →
API response →
Browser UI
```

Best low-risk removals: LLM enrichment, model-call logging, fuzzy empty-result recovery, chat-history context when no database is active, and database types you do not support. Each reduces capability, but has a contained blast radius.

Possible with a replacement: embeddings/RAG, answer generation, database management UI, or self-correction. Removing these without a replacement creates poor quality or a broken user path.

Core path: introspection, SQL generation, database execution, API chat endpoint, and an active database connection. Removing any of these changes the product from Text-to-SQL into something else.

Current architectural constraints

- SQL generation is explicitly SQL Server/T-SQL, although the connection manager advertises SQLite, PostgreSQL, MySQL/MariaDB, and MSSQL.
- The execution layer does not independently reject write statements. Prompts demand SELECT, but a code-level read-only validator is still absent.
- All users share one in-memory `active_db` and one `chat_history`. This is suitable for a local demo, not multi-user production.
- Database connection strings are returned by `GET /databases` and therefore reach the frontend. They may contain credentials.
- CORS allows every origin. Tightening it is important before deployment.

2. Application layers outside the six-stage pipeline

Frontend layer — `frontend/index.html`

A dependency-free single HTML file containing layout, styling, and JavaScript. It lists databases, connects/activates/removes them, displays the active database, sends chat requests, and renders simple Markdown-like bold/code/newlines. It calls a fixed `http://localhost:8000` API.

Inputs: user text and connection details. **Outputs:** REST calls and rendered responses. **State:** local `activeDb`, database list, and waiting indicator.

Removal impact: the API remains usable through Swagger or another client. Safe if the product will be API-only. The database sidebar can be removed separately if connections are configured server-side.

API/orchestration layer — `api/main.py`

Bootstraps settings, Groq, Ollama embeddings, the logger, database manager, and pipeline. It exposes health, database CRUD/activation, chat, and history-clear endpoints. It recognizes greetings and database-list questions, auto-selects a sole database, routes structural questions to a metadata shortcut, and otherwise runs the full pipeline.

Important behavior: database registrations and chat history disappear on restart. Chat history is only supplied to general LLM chat when no database is active; pipeline questions are effectively single-turn.

Removal impact: removing intent routing is low risk but greetings/list requests may become SQL attempts. Removing the API requires a different integration surface.

Configuration layer — `api/config.py` and `.env`

Loads the Groq key, model names, Ollama URL, log/cache paths, and enrichment controls. Startup fails without `GROQ_API_KEY`. Only enrichment currently has a feature flag.

Removal impact: hard-coding settings reduces deployability and risks secret exposure. Keep configuration even if individual settings are removed.

Database access layer — `api/database_manager.py`

Stores SQLAlchemy engines in memory, verifies connections with `SELECT 1`, disposes removed engines, exposes engine metadata, and executes SQL into a list of dictionaries. `pool_pre_ping=True` detects stale pooled connections.

Removal impact: core capability. It can be simplified to one preconfigured engine, which also lets you remove database CRUD and the sidebar.

Provider logging layer — `api/model_logging.py`

Wraps Groq chat and Ollama embeddings. Every call records timestamp, operation, request payload, latency, response/error, and token usage to JSONL. Vectors are not logged, but schema text, questions, SQL, results, and model answers can be logged.

Removal impact: model behavior is unchanged; auditability, cost analysis, and debugging decline. This is a good privacy-oriented removal if replaced with minimal redacted metrics.

3. Layer 1 — Introspection

File: `pipeline/introspection/layer.py`

This is the authoritative schema-discovery layer. SQLAlchemy inspects all non-system schemas. LangChain creates one `SQLDatabase` per schema with zero sample rows. For every table it collects columns, types, nullability, comments, primary keys, and foreign-key targets.

Output contract: `IntrospectionResult`

- `db_per_schema` and `schemas`: schema-level access.
- `table_metadata`: structured authoritative table/column metadata.
- Mutable enrichment fields used by layers 2 and 3: semantic descriptions, enriched documents, and sensitive-table flags.
- Helpers to resolve qualified names, list tables/columns, produce DDL, and yield one document per table.

The orchestrator keeps this result in memory by database name. Connecting/removing a database clears relevant in-memory caches. Structural questions such as “how many tables?” can be answered directly from this result without embeddings or model calls.

Dependencies: active SQLAlchemy engine, database inspection permissions, and LangChain `SQLDatabase`. **Downstream consumers:** every pipeline layer except final answer generation.

Can it be removed?

CORE / HIGH RISK Not in an unknown-database product. It could only be replaced by a static schema file, a curated semantic model, or user-supplied DDL. That replacement would remove “unknown database” discovery.

Features inside it that can be trimmed

Feature	Effect of removal	Risk
Multi-schema discovery	Simpler single-schema product; tables outside the chosen/default schema become invisible.	Medium
Foreign/primary-key metadata	Less metadata work, but joins and required structural columns become less reliable.	High
Metadata-question shortcut	Structural questions use embeddings + LLM + SQL instead of deterministic answers; slower and costlier.	Low functional / higher cost
LangChain <code>SQLDatabase</code>	Possible if DDL formatting is rebuilt directly from SQLAlchemy metadata.	Medium

4. Layer 2 — Enrichment

File: `pipeline/enrichment/layer.py`

Transforms raw identifiers and DDL into retrieval-friendly documents. Deterministic enrichment humanizes names, includes types/nullability/keys/references/comments, and detects potentially sensitive columns by name. Optional LLM enrichment asks Groq for conservative table and column meanings in bounded batches.

Caching and validation

Descriptions are stored per database under `.cache/enrichment/`. A DDL SHA-256 fingerprint and prompt version determine whether a table is reusable. Model output must be JSON, unknown table IDs are ignored, and only exact known column names are accepted. If an LLM batch fails, deterministic metadata remains available.

Already optional: set `LLM_ENRICHMENT_ENABLED=false`. The layer still produces deterministic documents and sensitive-name flags. This is the safest way to reduce first-query cost and avoid sending full schema metadata to Groq.

Can it be removed?

OPTIONAL WITH SMALL ADAPTER The RAG layer expects populated `table_metadata` and can use empty semantic descriptions. If the whole layer is bypassed, Layer 1 already supplies structured metadata, but retrieval documents become identifier/comment based. Confirm RAG receives its inputs and remove enrichment cache/config wiring.

Remove	Benefit	Cost
LLM descriptions only	Lower cost, latency, and external schema exposure	Lower semantic retrieval quality for cryptic names and Persian/English mismatch
Sensitive-name detection	Less code	Currently only a flag; removing it loses a future policy hook, not query behavior
Persistent enrichment cache	Less disk state	LLM enrichment repeats after restart and costs more
Entire layer	Simpler pipeline	Raw metadata retrieval quality; must adjust orchestration and validate RAG inputs

5. Layer 3 — RAG / schema selection

File: `pipeline/rag/layer.py`

Prevents the entire database schema from overflowing the chat model context. It builds embeddings for every table and column using local Ollama/BGE-M3, persists vectors under `.cache/embeddings/`, embeds each question, scores cosine similarity, selects tables and columns, and formats a compact schema prompt.

Selection behavior

- At most 8 tables; up to 24 rerank candidates; minimum similarity 0.30.
- An explicit `schema.table` mention is included directly.
- Base entities can outrank close derivative tables such as “Item/Detail/History” when qualifiers were not requested.
- Tables with 30 or fewer columns keep all columns. Wider tables keep at least the top 20, plus primary keys, foreign keys, and close identifier variants.
- A shared 21,000-character schema budget omits lower-priority columns fairly across selected tables.
- Embedding cache entries are keyed by text digest and model name; changed metadata is selectively re-embedded. Bad/NaN batches fall back to resilient single-document embedding.

Can it be removed?

HIGH RISK ON LARGE SCHEMAS For a small known schema, replace it with “send all DDL” or configured table allowlists. For unknown/wide databases, removal causes context overflow, cost/latency growth, and wrong-table selection. Removing only persistent vector caching preserves correctness but repeats expensive embedding work.

Ollama removal path: use lexical/BM25 matching, a hosted embedding provider, or explicit table selection. Simply deleting embeddings breaks `RAGLayer.run()`.

6. Layer 4 — SQL generation

File: `pipeline/sql_generation/layer.py`

Sends the natural-language question and compact schema to Groq. The prompt requires T-SQL, SELECT-only output, bracketed identifiers, TOP, Unicode N'...' literals, simple tables, and only necessary joins. The response parser removes Markdown fences, collapses duplicate SELECT tokens, and extracts the first SELECT/WITH expression.

Safety gap: extraction checks that SELECT or WITH appears, but does not prove the complete statement is read-only. A malicious or mistaken model response containing additional statements could reach database execution. Use a read-only database account and add AST/dialect validation before production.

Can it be removed?

CORE Only if users supply SQL themselves or if questions map to fixed query templates. It can be replaced with a different model or deterministic query builder.

Dialect mismatch

This layer always produces SQL Server syntax, while the database UI accepts SQLite, PostgreSQL, and MySQL. If those database types matter, make dialect a pipeline input and use dialect-specific prompts/validation. Otherwise, removing non-MSSQL connection options is a sensible simplification.

7. Layer 5 — execution and self-correction

File: `pipeline/self_correction/layer.py`

Executes generated SQL through the database manager and returns successful rows. On SQL errors, it asks Groq to repair the query using the error, original question, and selected schema, for up to three total attempts.

If valid SQL returns no rows, it parses up to three exact text equality predicates, confirms referenced columns are strings, probes at most 200 real values using bounded LIKE terms, scores Persian-aware normalized similarity, keeps up to five candidates above 0.70, and permits one model-assisted correction using only observed values. This helps spelling/script-variant mistakes without inventing replacements.

Can features be removed?

Feature	Outcome	Risk
Fuzzy empty-result recovery	Valid zero-row queries immediately return "No matching records."	Low; simplest contained removal
LLM error repair	First SQL error is returned/fails; lower model cost and fewer retries	Medium; worse success rate
All correction but keep execution	A thin executor returns first result/error	Medium
Execution	No data answer is possible	Core

8. Layer 6 — answer generation

File: `pipeline/answer_generation/layer.py`

If there are no rows, returns a fixed “No matching records” message without a model call. Otherwise it sends the question, executed SQL, and all rows serialized as JSON to Groq and requests a concise answer grounded only in those results.

Can it be removed?

OPTIONAL Return rows directly as JSON or render them as a table in the frontend. This eliminates a model call, reduces latency/cost, and avoids sending result data to Groq. The tradeoff is losing summaries, number formatting, and natural-language explanations.

Scaling issue: there is no row or character limit before result JSON is placed in the model prompt. Large result sets can exceed context or expose more data than necessary. Adding query limits and deterministic table rendering may be preferable to removing the layer outright.

9. Cross-cutting caches, privacy, and state

Component	Stored data	Removal effect
In-memory introspection cache	Schema objects and metadata per database	Repeated schema inspection on every question
Enrichment cache	Generated table/column descriptions and fingerprints	Repeated enrichment or metadata-only behavior
Embedding cache	Vectors, retrieval text, hashes, model metadata	Repeated local embedding after restarts
Model JSONL logs	Prompts, schema, questions, results, answers, timing, usage	Less audit/debug visibility; substantially less retained sensitive data
Chat history	One global in-memory conversation	Little impact on active-database pipeline questions

10. Feature-removal decision matrix

Candidate	Complexity	Main benefit	Main loss / required replacement
Disable LLM enrichment via env	None	Cost, latency, schema privacy	Weaker semantics for cryptic names
Remove fuzzy zero-row correction	Low	Less probing/model use; simpler behavior	Misspellings return no rows
Remove answer-generation LLM	Low-medium	Cost, latency, result privacy	Add JSON/table presentation
Remove full model-call payload logs	Low	Privacy and disk reduction	Use redacted metrics if observability matters
Remove non-MSSQL DB types	Low	Product truth matches T-SQL implementation	No SQLite/Postgres/MySQL connectivity
Single preconfigured database	Medium	Removes sidebar CRUD, active selection, credential exposure	Need secure server-side connection config
Remove LLM SQL error repair	Low	Predictability and lower cost	More failed questions
Remove embeddings/Ollama	Medium-high	No local model dependency	Need lexical retrieval, allowlist, or all-DDL strategy
Remove RAG entirely	High	Simpler architecture	Only viable for small schemas; context/quality risk
Remove introspection	High	No live schema discovery	Must supply static schema; no longer "unknown data"

Recommended simplification profiles

Privacy-first: disable LLM enrichment; replace answer generation with table/JSON output; redact or remove payload logging; use a read-only DB account; keep local embeddings.

Lowest operating complexity: support MSSQL only; preconfigure one database server-side; remove DB-management UI; disable fuzzy recovery; keep RAG and SQL generation.

Small-schema prototype: bypass enrichment and embeddings, send all DDL under a strict size limit, generate SQL once, execute once, return a frontend table. This is much smaller but should fail closed when schema size exceeds the limit.

11. Safe removal order and verification

1. **Define the product boundary:** supported database dialects, single vs multiple databases, hosted vs local models, and whether raw results are acceptable.
2. **Use existing flags first:** disable LLM enrichment without deleting code, then compare retrieval quality.
3. **Remove leaf features:** answer generation, fuzzy zero-row repair, verbose logging, greeting/history behavior. These have few downstream consumers.
4. **Replace before deleting shared infrastructure:** add lexical retrieval before removing embeddings; add static/configured schema before removing introspection.
5. **Test each cut:** structural question, single-table lookup, join, aggregate, Persian literal, misspelled value, invalid column, empty result, wide table, restart/cache reuse, and database removal/reconnection.

Minimum security work regardless of feature cuts

- Use database credentials restricted to read-only SELECT access.
- Add code-level SQL validation and reject multiple statements and data-changing operations.
- Stop returning connection strings to browsers; return only safe display metadata.
- Restrict CORS and add authentication/authorization before remote deployment.
- Cap query rows and answer-generation payload size.
- Move shared global state to per-user sessions if more than one user is possible.

12. Source map

Area	Primary source
Pipeline orchestration and metadata shortcut	<code>pipeline/__init__.py</code>
Shared layer dependency contract	<code>pipeline/base.py</code>
Six pipeline stages	<code>pipeline/*/layer.py</code>
HTTP API, intent routing, shared state	<code>api/main.py</code>
Connection lifecycle and execution	<code>api/database_manager.py</code>
Provider audit logging	<code>api/model_logging.py</code>
Environment settings	<code>api/config.py</code> , <code>.env.example</code>
Browser client	<code>frontend/index.html</code>

This guide describes the repository as reviewed on the preparation date. It is an architecture and change-impact document, not a guarantee of production security.